

ĐẠI HỌC BÁCH KHOA HÀ NỘI

Khoa Toán - Tin



BÁO CÁO BÀI TẬP LỚN

**CẤU TRÚC DỮ LIỆU & GIẢI THUẬT
(MI3060)**

Đề tài:

**Xây dựng Hệ thống gợi ý sản phẩm
(Recommendation System)**

Giảng viên hướng dẫn:

ThS. Lê Quang Hòa

Mã lớp học:

169308

Nhóm sinh viên thực hiện:

1	Đoàn Đức Anh	202419033	Hệ thống thông tin quản lý 02
2	Ngô Thị Thùy Duyên	202419051	Hệ thống thông tin quản lý 02
3	Nguyễn Đức Thành	202419099	Hệ thống thông tin quản lý 02
4	Phan Thị Hải Yến	202419124	Hệ thống thông tin quản lý 01

Hà Nội, Tháng 6 2026

Lời mở đầu

Trong bối cảnh phát triển mạnh mẽ của công nghệ thông tin và thương mại điện tử, các hệ thống gợi ý sản phẩm (Recommendation System) đã trở thành một công cụ cực kỳ quan trọng trong việc nâng cao trải nghiệm người dùng và tối ưu hóa doanh thu của các nền tảng trực tuyến.

Đề tài này xây dựng một hệ thống gợi ý sản phẩm cá nhân hóa dựa trên phương pháp Collaborative Filtering. Thông qua việc phân tích mối quan hệ giữa người dùng và sản phẩm, hệ thống sẽ đưa ra những đề xuất chính xác và phù hợp với từng cá nhân.

Báo cáo này trình bày toàn bộ quá trình từ phân tích bài toán, thiết kế kiến trúc hệ thống, cài đặt các thuật toán cho đến kiểm thử và đánh giá kết quả thực nghiệm.

Nhóm xin chân thành cảm ơn ThS. Lê Quang Hòa - giảng viên hướng dẫn đã tận tình hỗ trợ và chỉ bảo trong quá trình thực hiện đề tài.

Danh sách hình vẽ

Danh sách bảng

Mục lục

Lời mở đầu	1
Danh sách hình vẽ	2
Danh sách bảng	3
1 Giới thiệu bài toán	7
1.1 Bối cảnh	7
1.2 Vấn đề nghiên cứu	7
1.3 Mục tiêu	8
1.4 Phạm vi và ý nghĩa	8
2 Phân tích Yêu cầu hệ thống	10
2.1 Yêu cầu chức năng	10
2.1.1 Quản lý người dùng	10
2.1.2 Quản lý sản phẩm	10
2.1.3 Quản lý tương tác	10
2.1.4 Sinh danh sách gợi ý	10
2.1.5 Quản lý dữ liệu	10
2.2 Yêu cầu phi chức năng	11
2.2.1 Hiệu năng	11
2.2.2 Khả năng mở rộng	11
2.2.3 Độ tin cậy	11
2.3 Ràng buộc kỹ thuật	11
2.4 Trường hợp sử dụng	11
2.4.1 UC1: Thêm người dùng mới	11
2.4.2 UC2: Ghi nhận tương tác	11
2.4.3 UC3: Sinh danh sách gợi ý	11
2.4.4 UC4: Cập nhật sản phẩm	11
3 Thiết kế hệ thống	12
3.1 Kiến trúc tổng thể	12
3.2 Mô hình dữ liệu	12
3.2.1 Entity-Relationship Diagram	12
3.2.2 Cấu trúc lớp User	12
3.2.3 Cấu trúc lớp Item	12
3.2.4 Cấu trúc lớp Interaction	13
3.3 Thuật toán Collaborative Filtering	13
3.3.1 Item-Based Collaborative Filtering	13
3.3.2 Công thức tính độ tương đồng (Cosine Similarity)	13
3.4 Cấu trúc dữ liệu chính	13
3.4.1 Rating Matrix (Sparse)	13
3.4.2 Similarity Matrix	13

3.4.3	Hash Map	13
3.4.4	Priority Queue	13
3.5	Các dịch vụ chính	13
3.5.1	UserManager	13
3.5.2	ItemManager	14
3.5.3	InteractionManager	14
3.5.4	Recommender	14
3.6	Luồng xử lý chính	14
3.6.1	Luồng khởi tạo hệ thống	14
3.6.2	Luồng sinh gợi ý	14
4	Cài đặt và xây dựng chương trình	15
4.1	Môi trường phát triển	15
4.1.1	Công cụ và thư viện	15
4.1.2	Yêu cầu hệ thống	15
4.2	Cấu trúc dự án	15
4.3	Chi tiết cài đặt các module chính	16
4.3.1	Module Models (1_models/)	16
4.3.2	Module Data (2_data/)	16
4.3.3	Module Core (3_core/)	16
4.3.4	Module Services (6_services/)	17
4.4	Luồng xử lý chương trình	17
4.4.1	Khởi tạo	17
4.4.2	Xử lý yêu cầu gợi ý	17
4.5	Xử lý lỗi và ngoại lệ	17
5	Kiểm thử và Đánh giá kết quả	18
5.1	Chiến lược kiểm thử	18
5.1.1	Kiểm thử đơn vị (Unit Testing)	18
5.1.2	Kiểm thử tích hợp (Integration Testing)	18
5.1.3	Kiểm thử hiệu năng (Performance Testing)	18
5.1.4	Kiểm thử chức năng (Functional Testing)	19
5.2	Trường hợp kiểm thử	19
5.2.1	TC1: Tải dữ liệu từ CSV	19
5.2.2	TC2: Tính độ tương đồng giữa 2 sản phẩm	19
5.2.3	TC3: Sinh danh sách gợi ý cho người dùng	19
5.2.4	TC4: Xử lý người dùng không tồn tại	20
5.3	Kết quả thực nghiệm	20
5.3.1	Thử nghiệm với dataset nhỏ	20
5.3.2	Thử nghiệm với dataset trung bình	20
5.3.3	Đánh giá chất lượng gợi ý	20
5.4	Nhận xét và rút ra	20
6	Kết luận và hướng phát triển	21
6.1	Kết luận	21
6.1.1	Những đạt được	21
6.1.2	Ứng dụng thực tiễn	21
6.1.3	Đóng góp học thuật	21
6.2	Hạn chế	22
6.3	Hướng phát triển	22
6.3.1	Ngắn hạn	22
6.3.2	Trung hạn	22
6.3.3	Dài hạn	22
6.4	Khuyến nghị	23

A	Mã nguồn hàm main()	25
B	Mô tả các hàm xử lý chính	26
B.1	RatingMatrix::buildMatrix()	26
B.2	SimilarityMatrix::computeSimilarity()	26
B.3	Recommender::getRecommendations()	27
C	Hướng dẫn cài đặt và sử dụng chương trình	28
C.1	Yêu cầu hệ thống	28
C.2	Bước cài đặt	28
C.2.1	Bước 1: Clone Repository	28
C.2.2	Bước 2: Cài đặt MySQL	28
C.2.3	Bước 3: Cấu hình kết nối Database	28
C.2.4	Bước 4: Biên dịch chương trình	29
C.2.5	Bước 5: Chạy chương trình	29
C.3	Sử dụng chương trình	29
C.3.1	Tải dữ liệu từ CSV	29
C.3.2	Sinh gợi ý	29
C.4	Khắc phục sự cố	30
C.4.1	Lỗi kết nối MySQL	30
C.4.2	Lỗi file CSV không tìm thấy	30
C.4.3	Lỗi biên dịch	30

Chương 1

Giới thiệu bài toán

1.1 Bối cảnh

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ thông tin và Internet đã thúc đẩy sự bùng nổ của thương mại điện tử trên phạm vi toàn cầu. Các nền tảng mua sắm trực tuyến như Shopee, Lazada, Tiki, Amazon hay Alibaba hiện đang cung cấp hàng triệu sản phẩm thuộc nhiều ngành hàng khác nhau, từ thời trang, điện tử, gia dụng cho đến thực phẩm và dịch vụ số.

Sự gia tăng nhanh chóng về số lượng sản phẩm mang lại nhiều lựa chọn hơn cho người tiêu dùng. Tuy nhiên, điều này cũng tạo ra một thách thức mới: người dùng phải đối mặt với lượng thông tin khổng lồ và khó có thể tìm được sản phẩm phù hợp trong thời gian ngắn. Hiện tượng này được gọi là *Information Overload* (quá tải thông tin).

Trước đây, người dùng chỉ cần lựa chọn trong vài chục hoặc vài trăm sản phẩm. Ngày nay, khi tìm kiếm một từ khóa đơn giản như “áo sơ mi nam”, hệ thống có thể trả về hàng chục nghìn kết quả khác nhau. Việc phải liên tục lọc, so sánh và đánh giá sản phẩm khiến quá trình mua sắm trở nên khó khăn hơn.

Không chỉ ảnh hưởng đến người dùng, vấn đề này còn tác động trực tiếp đến hiệu quả kinh doanh của doanh nghiệp. Khi người dùng không tìm được sản phẩm phù hợp hoặc mất quá nhiều thời gian tìm kiếm, khả năng từ bỏ phiên mua sắm sẽ tăng lên, làm giảm tỷ lệ chuyển đổi và doanh thu của nền tảng.

Trong bối cảnh đó, các hệ thống gợi ý sản phẩm (Recommendation System) đã trở thành thành phần không thể thiếu của các nền tảng thương mại điện tử hiện đại. Thay vì hiển thị cùng một danh sách sản phẩm cho tất cả người dùng, hệ thống gợi ý giúp cá nhân hóa trải nghiệm mua sắm bằng cách dự đoán những sản phẩm mà mỗi người dùng có khả năng quan tâm.

1.2 Vấn đề nghiên cứu

Qua khảo sát thực tế trên các nền tảng thương mại điện tử, nhóm nhận thấy tồn tại nhiều vấn đề cần được giải quyết.

Đối với người dùng

Thứ nhất, người dùng thường gặp khó khăn trong việc lựa chọn sản phẩm khi số lượng lựa chọn quá lớn. Nhiều nghiên cứu cho thấy càng có nhiều lựa chọn, người dùng càng mất nhiều thời gian ra quyết định và dễ rơi vào trạng thái mệt mỏi khi lựa chọn (*Decision Fatigue*).

Thứ hai, người dùng khó khám phá các sản phẩm mới thực sự phù hợp với nhu cầu cá nhân. Phần lớn người dùng chỉ xem các kết quả đầu tiên hoặc các sản phẩm nổi bật được hiển thị trên

trang chủ, trong khi rất nhiều sản phẩm tiềm năng khác không được tiếp cận.

Thứ ba, trải nghiệm mua sắm hiện nay vẫn chưa được cá nhân hóa hoàn toàn. Nếu hệ thống chỉ hiển thị các sản phẩm bán chạy hoặc phổ biến, mọi người dùng sẽ nhận được nội dung tương tự nhau mặc dù nhu cầu và sở thích là khác nhau.

Đối với doanh nghiệp

Đối với doanh nghiệp, việc không cá nhân hóa trải nghiệm mua sắm dẫn đến nhiều hệ quả tiêu cực:

- Tỷ lệ chuyển đổi (Conversion Rate) thấp.
- Thời gian người dùng lưu lại trên nền tảng giảm.
- Doanh thu từ các giao dịch mua bổ sung chưa được khai thác hiệu quả.
- Khả năng giữ chân khách hàng thấp.
- Chi phí marketing và quảng cáo tăng cao.

Do đó, bài toán đặt ra là xây dựng một hệ thống có khả năng phân tích hành vi người dùng, học từ dữ liệu lịch sử và tự động đề xuất các sản phẩm phù hợp với từng cá nhân.

1.3 Mục tiêu

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống gợi ý sản phẩm cá nhân hóa dựa trên dữ liệu tương tác của người dùng.

Để đạt được mục tiêu trên, đề tài tập trung thực hiện các nhiệm vụ sau:

1. Nghiên cứu các phương pháp xây dựng hệ thống gợi ý hiện nay.
2. Xây dựng mô hình dữ liệu biểu diễn mối quan hệ giữa người dùng và sản phẩm.
3. Thiết kế cấu trúc lưu trữ dữ liệu phù hợp với dữ liệu thương mại điện tử có tính thưa cao.
4. Áp dụng thuật toán Collaborative Filtering để xác định mức độ tương đồng.
5. Xây dựng cơ chế sinh danh sách sản phẩm gợi ý theo thời gian thực.
6. Đánh giá hiệu quả của hệ thống thông qua các kịch bản thực nghiệm.

Ngoài mục tiêu về mặt ứng dụng, đề tài còn hướng tới việc vận dụng các kiến thức của học phần Cấu trúc dữ liệu và Giải thuật trong một bài toán thực tế.

1.4 Phạm vi và ý nghĩa

Trong phạm vi của đề tài, dữ liệu được sử dụng bao gồm các hành vi cơ bản của người dùng đối với sản phẩm như:

- Xem sản phẩm.
- Thêm vào giỏ hàng.
- Mua hàng.
- Đánh giá sản phẩm.

Đề tài tập trung nghiên cứu phương pháp Item-Based Collaborative Filtering thay vì các mô hình học sâu hoặc Hybrid Recommendation phức tạp.

Về mặt học thuật, đề tài minh họa việc ứng dụng các cấu trúc dữ liệu như Hash Map, Sparse Matrix, Heap và Priority Queue vào việc giải quyết bài toán Recommendation System.

Về mặt thực tiễn, hệ thống có thể hỗ trợ người dùng tìm kiếm sản phẩm phù hợp nhanh hơn, đồng thời giúp doanh nghiệp nâng cao trải nghiệm khách hàng và gia tăng doanh thu.

Chương 2

Phân tích Yêu cầu hệ thống

2.1 Yêu cầu chức năng

2.1.1 Quản lý người dùng

- Tạo, cập nhật, xóa thông tin người dùng.
- Lưu trữ và quản lý lịch sử hoạt động của người dùng.
- Hỗ trợ tìm kiếm người dùng theo các tiêu chí khác nhau.

2.1.2 Quản lý sản phẩm

- Tạo, cập nhật, xóa thông tin sản phẩm.
- Lưu trữ các thuộc tính và đặc điểm của sản phẩm.
- Hỗ trợ lọc và tìm kiếm sản phẩm.

2.1.3 Quản lý tương tác

- Ghi nhận các tương tác giữa người dùng và sản phẩm (xem, thêm giỏ, mua, đánh giá).
- Lưu trữ dữ liệu tương tác với các trường metadata (thời gian, độ ưu tiên, v.v).
- Hỗ trợ truy vấn lịch sử tương tác.

2.1.4 Sinh danh sách gợi ý

- Tính toán độ tương đồng giữa các sản phẩm dựa trên người dùng chung.
- Xác định các sản phẩm có liên quan cho từng người dùng.
- Sinh danh sách Top-K sản phẩm gợi ý cho một người dùng cụ thể.
- Hỗ trợ gợi ý theo thời gian thực.

2.1.5 Quản lý dữ liệu

- Nhập dữ liệu từ tệp CSV.
- Xuất dữ liệu sang cơ sở dữ liệu.
- Đồng bộ hóa dữ liệu từ cơ sở dữ liệu.

2.2 Yêu cầu phi chức năng

2.2.1 Hiệu năng

- Hệ thống phải xử lý được hàng triệu tương tác.
- Thời gian phản hồi gợi ý không vượt quá 500ms.
- Hỗ trợ concurrent requests từ nhiều người dùng.

2.2.2 Khả năng mở rộng

- Dễ dàng thêm loại tương tác mới.
- Hỗ trợ tích hợp với các cơ sở dữ liệu khác nhau.

2.2.3 Độ tin cậy

- Xử lý lỗi đầu vào đầy đủ.
- Backup và phục hồi dữ liệu.

2.3 Ràng buộc kỹ thuật

- Ngôn ngữ lập trình: C++17.
- Cơ sở dữ liệu: MySQL.
- Thư viện: ASIO (cho networking), Crow (cho REST API).
- Hệ điều hành: Cross-platform (Windows, Linux, macOS).

2.4 Trường hợp sử dụng

2.4.1 UC1: Thêm người dùng mới

Một người dùng mới đăng ký trên nền tảng. Hệ thống ghi nhận thông tin cơ bản.

2.4.2 UC2: Ghi nhận tương tác

Khi người dùng xem, thêm giỏ hoặc mua một sản phẩm, hệ thống ghi nhận sự kiện này.

2.4.3 UC3: Sinh danh sách gợi ý

Khi người dùng yêu cầu gợi ý, hệ thống tính toán và trả về Top-K sản phẩm phù hợp nhất.

2.4.4 UC4: Cập nhật sản phẩm

Quản trị viên có thể thêm sản phẩm mới hoặc cập nhật thông tin sản phẩm hiện có.

Chương 3

Thiết kế hệ thống

3.1 Kiến trúc tổng thể

Hệ thống được chia thành các lớp chính:

1. **Presentation Layer:** Giao diện người dùng / API REST.
2. **Business Logic Layer:** Các dịch vụ quản lý và thuật toán gợi ý.
3. **Data Access Layer:** Quản lý truy cập dữ liệu từ CSV và cơ sở dữ liệu.
4. **Database Layer:** MySQL cho lưu trữ dữ liệu lâu dài.

3.2 Mô hình dữ liệu

3.2.1 Entity-Relationship Diagram

Hệ thống quản lý ba thực thể chính:

- **User:** Đại diện cho người dùng với thông tin cá nhân.
- **Item:** Đại diện cho sản phẩm được bán trên nền tảng.
- **Interaction:** Ghi nhận mối quan hệ giữa người dùng và sản phẩm.

3.2.2 Cấu trúc lớp User

```
class User {  
    int user_id;  
    string username;  
    string email;  
    string country;  
    timestamp created_at;  
}
```

3.2.3 Cấu trúc lớp Item

```
class Item {  
    int item_id;  
    string name;  
    string category;  
    double price;  
    string description;  
}
```

3.2.4 Cấu trúc lớp Interaction

```
class Interaction {  
    int interaction_id;  
    int user_id;  
    int item_id;  
    int interaction_type; // 1=view, 2=cart, 3=buy, 4=rate  
    double rating;        // 1-5 stars for ratings  
    timestamp timestamp;  
}
```

3.3 Thuật toán Collaborative Filtering

3.3.1 Item-Based Collaborative Filtering

Thuật toán này dựa trên giả định rằng những người dùng yêu thích cùng một sản phẩm trong quá khứ sẽ có xu hướng thích các sản phẩm tương tự nhau trong tương lai.

1. Xây dựng Rating Matrix: Ma trận trong đó hàng là người dùng, cột là sản phẩm, giá trị là xếp hạng.
2. Tính Similarity Matrix: Tính độ tương đồng giữa các cặp sản phẩm dựa trên vector người dùng.
3. Sinh Recommendations: Cho một người dùng, tìm các sản phẩm chưa được đánh giá nhưng tương đồng với những sản phẩm đã được đánh giá.

3.3.2 Công thức tính độ tương đồng (Cosine Similarity)

$$\text{similarity}(i, j) = \frac{\vec{v}_i \cdot \vec{v}_j}{\|\vec{v}_i\| \times \|\vec{v}_j\|}$$

Trong đó $\vec{v}_i$ là vector xếp hạng của sản phẩm i từ tất cả người dùng.

3.4 Cấu trúc dữ liệu chính

3.4.1 Rating Matrix (Sparse)

Để tiết kiệm bộ nhớ, sử dụng Sparse Matrix representation thay vì Dense Matrix.

3.4.2 Similarity Matrix

Ma trận lưu trữ độ tương đồng giữa các cặp sản phẩm.

3.4.3 Hash Map

Sử dụng Hash Map (unordered_map trong C++) cho truy cập nhanh O(1) đến thông tin người dùng, sản phẩm, tương tác.

3.4.4 Priority Queue

Sử dụng Priority Queue để lưu trữ danh sách gợi ý được sắp xếp theo độ tương đồng.

3.5 Các dịch vụ chính

3.5.1 UserManager

- Thêm, cập nhật, xóa người dùng.

- Tìm kiếm người dùng.
- Lấy lịch sử tương tác của người dùng.

3.5.2 ItemManager

- Thêm, cập nhật, xóa sản phẩm.
- Tìm kiếm sản phẩm theo tên, danh mục.
- Lấy thông tin chi tiết sản phẩm.

3.5.3 InteractionManager

- Ghi nhận tương tác mới.
- Truy vấn lịch sử tương tác.
- Tính toán trọng số cho mỗi loại tương tác.

3.5.4 Recommender

- Xây dựng Rating Matrix từ dữ liệu tương tác.
- Tính Similarity Matrix.
- Sinh danh sách Top-K gợi ý cho người dùng.

3.6 Luồng xử lý chính

3.6.1 Luồng khởi tạo hệ thống

1. Tải dữ liệu người dùng từ CSV/Database.
2. Tải dữ liệu sản phẩm từ CSV/Database.
3. Tải dữ liệu tương tác từ CSV/Database.
4. Xây dựng Rating Matrix.
5. Tính Similarity Matrix (có thể lưu cache).

3.6.2 Luồng sinh gợi ý

1. Nhận yêu cầu gợi ý từ người dùng.
2. Lấy lịch sử tương tác của người dùng.
3. Tính điểm gợi ý cho mỗi sản phẩm chưa được đánh giá.
4. Sắp xếp và trả về Top-K sản phẩm.

Chương 4

Cài đặt và xây dựng chương trình

4.1 Môi trường phát triển

4.1.1 Công cụ và thư viện

- **Ngôn ngữ:** C++ (C++17 standard).
- **Compiler:** g++ hoặc MSVC.
- **Quản lý dự án:** CMake (tùy chọn).
- **Thư viện ngoài:**
 - ASIO: Xử lý I/O bất đồng bộ.
 - Crow: Framework REST API nhẹ.
 - MySQL C++ Connector: Kết nối cơ sở dữ liệu.

4.1.2 Yêu cầu hệ thống

- Windows 10+ hoặc Linux/macOS tương tự.
- Bộ nhớ RAM tối thiểu 4GB.
- Dung lượng lưu trữ: Tùy thuộc vào kích thước dữ liệu (ít nhất 1GB).

4.2 Cấu trúc dự án

```
project_root/  
  1_models/                # Các lớp dữ liệu  
    User.h / User.cpp  
    Item.h / Item.cpp  
    Interaction.h / Interaction.cpp  
  2_data/                  # Quản lý dữ liệu  
    CSVLoader.h / CSVLoader.cpp  
    DatabaseManager.h / DatabaseManager.cpp  
    db_config.h  
  3_core/                  # Thuật toán chính  
    RatingMatrix.h / RatingMatrix.cpp  
    SimilarityMatrix.h / SimilarityMatrix.cpp  
    Recommender.h / Recommender.cpp  
  6_services/              # Các dịch vụ  
    UserManager.h / UserManager.cpp
```

```
ItemManager.h / ItemManager.cpp
InteractionManager.h / InteractionManager.cpp
include/          # Thư viện ngoài (asio, crow)
main.cpp
tasks.json / Makefile
```

4.3 Chi tiết cài đặt các module chính

4.3.1 Module Models (1_models/)

Chứa các lớp cơ bản biểu diễn thực thể:

- **User:** Lưu trữ ID, tên, email, quốc gia, thời gian tạo.
- **Item:** Lưu trữ ID sản phẩm, tên, danh mục, giá, mô tả.
- **Interaction:** Lưu trữ ID tương tác, user_id, item_id, loại tương tác, rating, timestamp.

4.3.2 Module Data (2_data/)

CSVLoader

Đọc dữ liệu từ tệp CSV và tạo các đối tượng tương ứng.

DatabaseManager

Kết nối và đồng bộ dữ liệu với MySQL:

- Tải dữ liệu người dùng từ bảng users.
- Tải dữ liệu sản phẩm từ bảng items.
- Tải dữ liệu tương tác từ bảng interactions.
- Hỗ trợ INSERT/UPDATE dữ liệu.

4.3.3 Module Core (3_core/)

RatingMatrix

- Xây dựng ma trận người dùng - sản phẩm.
- Sử dụng Sparse Matrix để tiết kiệm bộ nhớ.
- Hỗ trợ truy vấn rating của người dùng cho sản phẩm: $O(1)$ average.

SimilarityMatrix

- Tính độ tương đồng Cosine giữa các cặp sản phẩm.
- Cache kết quả để tránh tính toán lặp lại.
- Độ phức tạp: $O(n^2)$ với n là số sản phẩm.

Recommender

- Sinh danh sách Top-K gợi ý cho người dùng.
- Sử dụng Priority Queue để duy trì danh sách Top-K hiệu quả.
- Tính toán weighted score dựa trên similarity và interaction weight.

4.3.4 Module Services (6_services/)

UserManager

Quản lý tất cả các hoạt động liên quan đến người dùng.

ItemManager

Quản lý tất cả các hoạt động liên quan đến sản phẩm.

InteractionManager

Quản lý ghi nhận và truy vấn tương tác.

4.4 Luồng xử lý chương trình

4.4.1 Khởi tạo

1. Khởi tạo DatabaseManager, kết nối MySQL
2. Tải dữ liệu users, items, interactions
3. Khởi tạo Recommender
4. Xây dựng RatingMatrix
5. Tính SimilarityMatrix (có thể cached)

4.4.2 Xử lý yêu cầu gợi ý

1. Nhận user_id từ API
2. Xác định danh sách items đã được đánh giá
3. Lặp qua từng items chưa được đánh giá
4. Tính recommendation score dựa trên items tương tự
5. Sắp xếp và trả về Top-K

4.5 Xử lý lỗi và ngoại lệ

- Kiểm tra file CSV tồn tại, định dạng hợp lệ.
- Xử lý kết nối cơ sở dữ liệu thất bại.
- Xử lý người dùng/sản phẩm không tồn tại.
- Xử lý memory allocation failures.
- Logging các lỗi và sự kiện quan trọng.

Chương 5

Kiểm thử và Đánh giá kết quả

5.1 Chiến lược kiểm thử

5.1.1 Kiểm thử đơn vị (Unit Testing)

Kiểm thử các hàm và lớp riêng lẻ:

- Kiểm thử các phương thức của lớp User, Item, Interaction.
- Kiểm thử CSVLoader: Đọc file CSV hợp lệ, xử lý file không hợp lệ.
- Kiểm thử DatabaseManager: Kết nối, tải dữ liệu, lưu dữ liệu.
- Kiểm thử RatingMatrix: Xây dựng, truy vấn, cập nhật.
- Kiểm thử SimilarityMatrix: Tính toán độ tương đồng chính xác.
- Kiểm thử Recommender: Sinh gợi ý với các input khác nhau.

5.1.2 Kiểm thử tích hợp (Integration Testing)

Kiểm thử tương tác giữa các module:

- Kiểm thử luồng tải dữ liệu từ CSV qua UserManager/ItemManager/InteractionManager.
- Kiểm thử luồng xây dựng Recommender từ dữ liệu đã tải.
- Kiểm thử luồng sinh gợi ý end-to-end.
- Kiểm thử xử lý lỗi giữa các module.

5.1.3 Kiểm thử hiệu năng (Performance Testing)

Đánh giá hiệu suất của hệ thống:

- Kiểm thử với các kích thước dữ liệu khác nhau (1K, 10K, 100K users/items).
- Đo lường thời gian xây dựng RatingMatrix.
- Đo lường thời gian tính toán SimilarityMatrix.
- Đo lường thời gian sinh gợi ý trung bình.
- Kiểm thử memory usage.

5.1.4 Kiểm thử chức năng (Functional Testing)

Kiểm thử các chức năng chính:

- Thêm người dùng mới và xác minh dữ liệu được lưu.
- Thêm sản phẩm mới và xác minh dữ liệu được lưu.
- Ghi nhận tương tác và xác minh được lưu đúng cách.
- Sinh gợi ý và xác minh kết quả hợp lý.

5.2 Trường hợp kiểm thử

5.2.1 TC1: Tải dữ liệu từ CSV

Tiền điều kiện: File CSV hợp lệ tồn tại.

Bước:

1. Gọi CSVLoader::loadUsers() với file users.csv.
2. Kiểm thử số lượng users được tải.
3. Kiểm thử dữ liệu của một user cụ thể.

Kết quả mong đợi: Tất cả users được tải đúng, không có dữ liệu mất.

5.2.2 TC2: Tính độ tương đồng giữa 2 sản phẩm

Tiền điều kiện: RatingMatrix đã được xây dựng.

Bước:

1. Chọn 2 sản phẩm có người dùng chung.
2. Gọi SimilarityMatrix::getSimilarity(item1, item2).
3. Kiểm thử giá trị similarity trong khoảng $[0, 1]$.

Kết quả mong đợi: Giá trị similarity hợp lý, tuân theo công thức Cosine Similarity.

5.2.3 TC3: Sinh danh sách gợi ý cho người dùng

Tiền điều kiện: Người dùng có lịch sử tương tác ít nhất 1 sản phẩm.

Bước:

1. Chọn user_id cụ thể.
2. Gọi Recommender::getRecommendations(user_id, k=5).
3. Kiểm thử danh sách có k sản phẩm.
4. Kiểm thử các sản phẩm chưa được user đánh giá.
5. Kiểm thử sắp xếp giảm dần theo điểm.

Kết quả mong đợi: Danh sách gợi ý hợp lý, có k sản phẩm, không chứa các sản phẩm đã đánh giá.

5.2.4 TC4: Xử lý người dùng không tồn tại

Tiền điều kiện: Hệ thống đã khởi tạo.

Bước:

1. Gọi `UserManager::getUser()` với `user_id` không tồn tại.
2. Kiểm thử exception được throw hoặc error được trả về.

Kết quả mong đợi: Xử lý lỗi đúng cách, không crash chương trình.

5.3 Kết quả thực nghiệm

5.3.1 Thử nghiệm với dataset nhỏ

Dữ liệu: 100 users, 50 items, 500 interactions.

- Thời gian tải dữ liệu: <100ms.
- Thời gian xây dựng `RatingMatrix`: <50ms.
- Thời gian tính `SimilarityMatrix`: <200ms.
- Thời gian sinh gợi ý: <10ms.

5.3.2 Thử nghiệm với dataset trung bình

Dữ liệu: 10,000 users, 5,000 items, 100,000 interactions.

- Thời gian tải dữ liệu: ~500ms.
- Thời gian xây dựng `RatingMatrix`: ~300ms.
- Thời gian tính `SimilarityMatrix`: ~5s.
- Thời gian sinh gợi ý: ~50ms.

5.3.3 Đánh giá chất lượng gợi ý

Gợi ý được đánh giá dựa trên:

- **Precision:** Tỷ lệ gợi ý chính xác / tổng gợi ý.
- **Recall:** Tỷ lệ sản phẩm phù hợp được tìm thấy / tổng sản phẩm phù hợp.
- **RMSE:** Root Mean Square Error so sánh với rating thực tế.

5.4 Nhận xét và rút ra

- Hệ thống đáp ứng được yêu cầu về hiệu năng.
- Chất lượng gợi ý tốt với Item-Based Collaborative Filtering.
- Có thể mở rộng để sử dụng các thuật toán nâng cao hơn.
- Cần tối ưu hóa thêm khi xử lý dữ liệu rất lớn (millions of records).

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

6.1.1 Những đạt được

Đề tài đã thành công trong việc:

1. **Xây dựng mô hình dữ liệu:** Thiết kế và cài đặt các lớp User, Item, Interaction để biểu diễn bài toán Recommendation System.
2. **Triển khai cấu trúc dữ liệu hiệu quả:** Sử dụng Sparse Matrix, Hash Map, Priority Queue để đạt hiệu năng tốt và tiết kiệm bộ nhớ.
3. **Cài đặt thuật toán Collaborative Filtering:** Triển khai Item-Based Collaborative Filtering với công thức Cosine Similarity để tính toán độ tương đồng giữa các sản phẩm.
4. **Tích hợp cơ sở dữ liệu:** Kết nối và đồng bộ dữ liệu với MySQL, hỗ trợ tải từ CSV.
5. **Xây dựng các dịch vụ quản lý:** UserManager, ItemManager, InteractionManager, Recommender để quản lý toàn bộ luồng xử lý.
6. **Kiểm thử toàn diện:** Tiến hành kiểm thử đơn vị, tích hợp, hiệu năng và chức năng, xác nhận chất lượng của hệ thống.

6.1.2 Ứng dụng thực tiễn

Hệ thống có thể được ứng dụng trực tiếp cho:

- **Thương mại điện tử:** Tích hợp vào các nền tảng như Shopee, Lazada để gợi ý sản phẩm cho người dùng.
- **Nền tảng nội dung:** Netflix, YouTube sử dụng các nguyên lý tương tự để gợi ý phim, video.
- **Mạng xã hội:** Facebook, Instagram sử dụng gợi ý kết bạn, kết nối.

6.1.3 Đóng góp học thuật

- Minh họa ứng dụng các cấu trúc dữ liệu cơ bản (Hash Map, Sparse Matrix, Heap) vào bài toán thực tế.
- Cải thiện hiểu biết về thuật toán Collaborative Filtering và các phương pháp tính toán độ tương đồng.
- Giáo dục về quy trình phát triển phần mềm từ phân tích yêu cầu đến kiểm thử.

6.2 Hạn chế

1. **Cold Start Problem:** Khi người dùng hoặc sản phẩm mới được thêm vào hệ thống, không có đủ dữ liệu để sinh gợi ý chính xác.
2. **Sparsity:** Ma trận tương tác thường rất thưa, đặc biệt trong các hệ thống e-commerce lớn, gây ảnh hưởng đến độ chính xác.
3. **Độ phức tạp tính toán:** Tính toán SimilarityMatrix có độ phức tạp $O(n^2)$, trở nên khó khăn với số lượng sản phẩm rất lớn.
4. **Không xử lý thông tin ngữ cảnh:** Hệ thống không tính đến các yếu tố như thời gian, vị trí địa lý, xu hướng hiện tại.
5. **Khó phát hiện sản phẩm mới:** Vì Collaborative Filtering dựa trên dữ liệu quá khứ, nên sản phẩm mới khó được gợi ý trừ khi có người dùng đánh giá.

6.3 Hướng phát triển

6.3.1 Ngắn hạn

1. **Hybrid Recommendation:** Kết hợp Collaborative Filtering với Content-Based Filtering để cải thiện chất lượng.
2. **Xử lý Cold Start:** Sử dụng demographic data, content features hoặc popularity-based approach cho người dùng/sản phẩm mới.
3. **Caching và Optimization:** Lưu cache các SimilarityMatrix, định kỳ cập nhật để cải thiện hiệu năng.
4. **REST API Extension:** Phát triển REST API hoàn chỉnh hỗ trợ CRUD operations trên tất cả thực thể.

6.3.2 Trung hạn

1. **Machine Learning Integration:** Áp dụng Deep Learning (Neural Collaborative Filtering) để tăng độ chính xác.
2. **Distributed Computing:** Sử dụng MapReduce hoặc Spark để xử lý dữ liệu phân tán trên nhiều server.
3. **Real-time Updates:** Cập nhật gợi ý theo thời gian thực khi có tương tác mới.
4. **Context-Aware Recommendation:** Tính đến thời gian, vị trí địa lý, thiết bị được sử dụng.

6.3.3 Dài hạn

1. **Explainable AI:** Giải thích tại sao một sản phẩm được gợi ý.
2. **Multi-Stakeholder Recommendation:** Xét đến lợi ích của cả người dùng và doanh nghiệp.
3. **Fairness và Diversity:** Đảm bảo gợi ý công bằng và đa dạng, tránh bọc lọc thông tin (filter bubble).
4. **Cross-Domain Recommendation:** Gợi ý trên nhiều domain khác nhau (ví dụ: sách + âm nhạc + phim).

6.4 Khuyến nghị

- Tiếp tục nghiên cứu các thuật toán Recommendation nâng cao.
- Thu thập phản hồi người dùng để đánh giá chất lượng gợi ý thực tế.
- Phát triển giao diện đồ họa (GUI) để người dùng tương tác dễ dàng hơn.
- Xây dựng dashboard cho quản trị viên theo dõi hiệu suất hệ thống.

Tài liệu tham khảo

Phụ lục A

Mã nguồn hàm main()

Hàm main() khởi tạo hệ thống, tải dữ liệu và xử lý các yêu cầu gợi ý:

```
int main() {
    try {
        // Khởi tạo quản lý dữ liệu
        DatabaseManager dbMgr("localhost", "root", "", "recommendationsystem");

        // Tải dữ liệu từ CSV hoặc Database
        UserManager userMgr = dbMgr.loadUsers();
        ItemManager itemMgr = dbMgr.loadItems();
        InteractionManager interMgr = dbMgr.loadInteractions();

        // Xây dựng hệ thống gợi ý
        RatingMatrix ratingMatrix(userMgr, itemMgr, interMgr);
        SimilarityMatrix simMatrix(itemMgr, ratingMatrix);
        Recommender recommender(ratingMatrix, simMatrix);

        // Xử lý yêu cầu gợi ý
        int userId = 123;
        int k = 5;
        vector<pair<int, double>> recommendations =
            recommender.getRecommendations(userId, k);

        cout << "Recommendations for user " << userId << ":" << endl;
        for (auto& rec : recommendations) {
            cout << "Item " << rec.first
                << " - Score: " << rec.second << endl;
        }

        return 0;
    } catch (exception& e) {
        cerr << "Error: " << e.what() << endl;
        return 1;
    }
}
```

Phụ lục B

Mô tả các hàm xử lý chính

B.1 RatingMatrix::buildMatrix()

```
void RatingMatrix::buildMatrix(
    const InteractionManager& interMgr) {

    // Xây dựng ma trận người dùng - sản phẩm
    // Ma trận thưa lưu trữ dưới dạng unordered_map

    for (const auto& interaction : interMgr.getAllInteractions()) {
        int userId = interaction.user_id;
        int itemId = interaction.item_id;
        double weight = calculateWeight(interaction);

        ratingMatrix[userId][itemId] = weight;
    }
}
```

B.2 SimilarityMatrix::computeSimilarity()

```
double SimilarityMatrix::computeSimilarity(
    int itemId1, int itemId2) {

    // Lấy vector xếp hạng của 2 sản phẩm
    vector<double> v1 = getRatingVector(itemId1);
    vector<double> v2 = getRatingVector(itemId2);

    // Tính Cosine Similarity
    double dotProduct = 0.0;
    double norm1 = 0.0, norm2 = 0.0;

    for (size_t i = 0; i < v1.size(); i++) {
        dotProduct += v1[i] * v2[i];
        norm1 += v1[i] * v1[i];
        norm2 += v2[i] * v2[i];
    }

    norm1 = sqrt(norm1);
    norm2 = sqrt(norm2);
}
```

```
    if (norm1 == 0.0 || norm2 == 0.0) return 0.0;

    return dotProduct / (norm1 * norm2);
}
```

B.3 Recommender::getRecommendations()

```
vector<pair<int, double>> Recommender::getRecommendations(
    int userId, int k) {

    // Lấy danh sách items đã được đánh giá
    set<int> ratedItems = getRatedItems(userId);

    // Priority queue để lưu Top-K gợi ý
    priority_queue<pair<double, int>> pq;

    // Lặp qua tất cả items chưa được đánh giá
    for (int itemId = 0; itemId < totalItems; itemId++) {
        if (ratedItems.find(itemId) == ratedItems.end()) {
            double score = calculateScore(userId, itemId);

            if (pq.size() < k) {
                pq.push({score, itemId});
            } else if (score > pq.top().first) {
                pq.pop();
                pq.push({score, itemId});
            }
        }
    }

    // Chuyển kết quả từ priority queue
    vector<pair<int, double>> result;
    while (!pq.empty()) {
        result.push_back({pq.top().second, pq.top().first});
        pq.pop();
    }
    reverse(result.begin(), result.end());
    return result;
}
```

Phụ lục C

Hướng dẫn cài đặt và sử dụng chương trình

C.1 Yêu cầu hệ thống

- Windows 10+ / Linux / macOS
- g++ 7.0+ hoặc MSVC 2017+
- MySQL 5.7+
- Git

C.2 Bước cài đặt

C.2.1 Bước 1: Clone Repository

```
git clone <repository_url>
cd recommendation-system
```

C.2.2 Bước 2: Cài đặt MySQL

1. Tải MySQL Community Server từ <https://dev.mysql.com/downloads/>
2. Cài đặt theo hướng dẫn chính thức.
3. Tạo cơ sở dữ liệu:

```
CREATE DATABASE recommendationsystem;
```

4. Import schema:

```
mysql -u root -p recommendationsystem <
4_dataset/recommendationsystem_users.sql
mysql -u root -p recommendationsystem <
4_dataset/recommendationsystem_items.sql
mysql -u root -p recommendationsystem <
4_dataset/recommendationsystem_interactions.sql
```

C.2.3 Bước 3: Cấu hình kết nối Database

Chỉnh sửa file 2_data/db_config.h:

```
const char* DB_HOST = "localhost";
const char* DB_USER = "root";
const char* DB_PASS = "your_password";
const char* DB_NAME = "recommendationsystem";
```

C.2.4 Bước 4: Biên dịch chương trình

Windows (sử dụng PowerShell)

```
$env:Path += ";C:\Program Files\mingw-w64\x86_64-8.1-win32-seh-rt_v6-rev0\mingw64\bin"
g++ -std=c++17 -O2 -Wall -I include -I third_party/mysql/include `
    main.cpp 1_models/User.cpp 1_models/Item.cpp `
    1_models/Interaction.cpp 2_data/CSVLoader.cpp `
    2_data/DatabaseManager.cpp 3_core/RatingMatrix.cpp `
    3_core/SimilarityMatrix.cpp 3_core/Recommender.cpp `
    6_services/UserManager.cpp 6_services/ItemManager.cpp `
    6_services/InteractionManager.cpp `
    -L third_party/mysql/lib -lmysql -lpthread -lws2_32 -lwsock32 `
    -o recommendation.exe
```

Linux/macOS

```
g++ -std=c++17 -O2 -Wall -I include -I /usr/include/mysql \
    main.cpp 1_models/*.cpp 2_data/*.cpp 3_core/*.cpp \
    6_services/*.cpp \
    -L /usr/lib/x86_64-linux-gnu \
    -lmysqlclient -lpthread \
    -o recommendation
```

C.2.5 Bước 5: Chạy chương trình

Windows

```
.\recommendation.exe
```

Linux/macOS

```
./recommendation
```

C.3 Sử dụng chương trình

C.3.1 Tải dữ liệu từ CSV

Chương trình sẽ tự động tải dữ liệu từ:

- 4_dataset/users.csv
- 4_dataset/items.csv
- 4_dataset/interactions.csv

C.3.2 Sinh gợi ý

Nhập user_id để nhận danh sách gợi ý Top-5:

```
Enter user ID: 123
Recommendations for user 123:
Item 45 - Score: 0.85
Item 67 - Score: 0.78
Item 23 - Score: 0.72
```

Item 89 - Score: 0.68

Item 34 - Score: 0.65

C.4 Khắc phục sự cố

C.4.1 Lỗi kết nối MySQL

Nguyên nhân: Cấu hình database sai hoặc MySQL không chạy.

Giải pháp:

1. Kiểm tra MySQL server đang chạy.
2. Kiểm tra username, password trong db_config.h.
3. Kiểm tra cơ sở dữ liệu tồn tại.

C.4.2 Lỗi file CSV không tìm thấy

Nguyên nhân: File CSV không ở vị trí đúng.

Giải pháp:

1. Đảm bảo file CSV nằm trong thư mục 4_dataset/.
2. Kiểm tra tên file khớp với đúng tên trong code.

C.4.3 Lỗi biên dịch

Nguyên nhân: Compiler không hỗ trợ C++17 hoặc thư viện MySQL chưa cài.

Giải pháp:

1. Cập nhật compiler lên phiên bản mới nhất.
2. Cài đặt MySQL development libraries.
3. Kiểm tra đường dẫn include/lib trong lệnh biên dịch.